

Ex. No. 8	Use Steg-Expose to detect hidden data in images
Date	

Description

Steg Expose is a tool used to detect hidden data in images using steganography analysis. It works by evaluating the statistical properties of an image to estimate whether any hidden data is embedded. Below is a step-by-step guide on how to use Steg Expose to detect hidden data in images:

Prerequisites:

1. **Java Runtime Environment (JRE):** Steg Expose is a Java-based tool, so ensure you have JRE installed.
2. **Steg Expose Tool:** Download Steg Expose from the [official GitHub repository](#).

Step-by-Step Process:

1. Download and Set Up Steg Expose:

- **Download the tool:** Go to [Steg Expose GitHub](#) and download the .jar file.
- **Install Java:** Ensure that Java is installed on your machine. You can download it from the official site.
- **Prepare environment:** Place the .jar file in a folder where you plan to work on the image files.

2. Select Images for Analysis:

- Collect the images you suspect might have hidden data. StegExpose supports formats such as .png, .jpg, .bmp, etc.

3. Open Command Line or Terminal:

- Navigate to the folder where you have saved the StegExpose.jar file.
- Open your **Command Prompt** (Windows) or **Terminal** (Linux/macOS).

4. Run Steg Expose on an Image:

- Use the following command to check a single image for hidden data:

```
bash
```

```
java -jar StegExpose.jar <image_file_path>
```

Example:

```
bash
```

```
java -jar StegExpose.jar test_image.png
```

- Replace <image_file_path> with the actual path of the image you want to check.

5. Analyze the Output:

- After running the tool, it will analyze the image and provide a score.
- **Output Explanation:**
 - StegExpose calculates a "suspect" score (between 0 and 1). The higher the score, the more likely that hidden data is present.
 - **Threshold values** (suggested by the tool):
 - **Less than 0.2:** Image is clean (no hidden data).
 - **0.2 - 0.3:** Possibly some hidden data.
 - **Above 0.3:** Likely that steganography is present.

6. Batch Analysis (Multiple Images):

- If you have multiple images to check, you can analyze them in bulk by specifying a folder. For example:

```
bash
```

```
java -jar StegExpose.jar <folder_path>
```

- Replace <folder_path> with the path to the folder containing the images.

7. Advanced Options (Optional):

- StegExpose provides additional parameters, such as adjusting the sensitivity of detection and output verbosity. Use --help to see the list of options:

```
bash
```

```
java -jar StegExpose.jar --help
```

8. Review the Results:

- For each image analyzed, review the scores and determine whether you need to investigate further.

Example:

```
bash
```

```
java -jar StegExpose.jar suspect_image.png
```

Output:

```
makefile
```

```
Analyzing suspect_image.png...
```

```
Result: 0.4
```

```
Steganography likely present
```

This output indicates that hidden data is likely present in the image suspect_image.png.

Output Screenshots:

```

Command Prompt
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\chandu> java -version
openjdk version "25.0.4.1" 2026-08-18 LTS
OpenJDK Runtime Environment Temurin-25.0.4.1+1 (build 25.0.4.1+1-LTS)
OpenJDK 64-Bit Server VM Temurin-25.0.4.1+1 (build 25.0.4.1+1-LTS, mixed mode, sharing)
PS C:\Users\chandu>

```

2	File name	Above steg	Secret mes	Primary Se	Chi Square	Sample Pa	RS analysis	Fusion (mean)
3	image 1.pr	FALSE	1370	0.003522	0	null	0.016585	0.006703
4	image 2.pr	FALSE	5271	0.050289	1.65E-14	null	0.02748	0.025923
5	image 3.pr	FALSE	863	0.012388	0	0.00537	0.016264	0.008506
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								
16								

```

PS C:\StegExposeLab> Start-Process .\results.csv
PS C:\StegExposeLab> Get-Content .\results.csv

File name,Above stego threshold?,Secret message size in bytes (ignore for clean files),Primary Sets,Chi Square,Sample Pairs,RS analysis,Fusion (mean)
image 1.png,false,1370,0.003522431921441729,0.0,null,0.016585235152236963,0.0067025556912262306
image 2.png,false,5271,0.050289071306961974,1.649152512663717E-14,null,0.027479554543971343,0.02592287528364994
image 3.png,false,863,0.012387543663056765,0.0,0.0053704199757471675,0.016264357216348613,0.008505580213788135
PS C:\StegExposeLab>

```

```
PS C:\Users\dhani> cd C:\StegExposeLab
PS C:\StegExposeLab> Get-ChildItem

Directory: C:\StegExposeLab
```

Mode	LastWriteTime		Length	Name
d----	19-09-2026	16:30		images
-a----	19-09-2026	16:21	3060550	StegExpose.jar

```
PS C:\StegExposeLab> Get-ChildItem .\images
```

```
Directory: C:\StegExposeLab\images
```

Mode	LastWriteTime		Length	Name
-a----	19-09-2026	16:30	613380	image 1.png
-a----	19-09-2026	16:30	610003	image 2.png
-a----	19-09-2026	16:30	304330	image 3.png

```
PS C:\StegExposeLab> java -jar .\StegExpose.jar .\images
PS C:\StegExposeLab> java -jar .\StegExpose.jar .\images default default results.csv
PS C:\StegExposeLab> Get-ChildItem
```

```
Directory: C:\StegExposeLab
```

Mode	LastWriteTime		Length	Name
d----	19-09-2026	16:30		images
-a----	19-09-2026	16:34	475	results.csv
-a----	19-09-2026	16:21	3060550	StegExpose.jar

Rubrics:

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1.GitHub Activity & Submission Regularity	3	
2.Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

Result:

Steg Expose successfully analyzed the image and generated a steganography detection score. The result indicated whether hidden data was likely present, demonstrating the use of Steg Expose for image-based forensic analysis.